

TraceWasm

The machine is a
parameter, not a fixture.

A WebAssembly runtime built so that
every component a guest program
can observe is something you supply.

Innovations in Compiler Technology (IICT) 2026

A compiler workshop bridging academia and industry · Indian Institute of Science,
Bengaluru · 2–3 October 2026 · compilertech.org

TraceWasm is a WebAssembly engine written from scratch in Rust. It ships two independent execution machines — a stack VM and a register VM — over one shared frontend, and an LLVM IR construction layer for an ahead-of-time tier. What makes it unusual is not either machine. It is that the memory, the host boundary, and the machine itself are all type parameters, so a guest program can be run against a described machine rather than a real one.

50,070	550	2
LINES OF RUST	TESTS	EXECUTION MACHINES
8 B	1.64×	3.55×
REGISTER INSTRUCTION	REGISTER VS STACK	SMALLER INSTR STREAM

Every profiler watches the machine. None of them watch the program.

The problem statement, and why it is an architecture problem rather than a tooling problem.

Ask a normal toolchain what a program did to memory and you get an approximation. `perf` samples the CPU at whatever interval the kernel permits and attributes cache misses to instruction pointers after the fact. Valgrind and Cachegrind get closer, at a 20–50× slowdown that changes the timing behaviour of the thing being measured. Instrumentation passes rewrite the program, so what runs is no longer the program you shipped.

All three share a defect: they observe `the machine` from outside and infer the program. The resolution is set by the operating system's willingness to be interrupted, not by the question being asked.

If you own the machine
the program runs on,
nothing has to be inferred.

That is the inversion. In an interpreter you control, every memory access, every call, every branch and every allocation already passes through code you wrote. The information is not hard to collect — it is sitting in the execution path. The only question is whether the architecture lets you put something of your own there. In most engines it does not: memory is a `Vec<u8>` welded into the instance, and the only extension point is a host-function ABI.

WebAssembly is the substrate that makes a complete account achievable

Why the target matters: a bounded, explicit address space with no ambient authority.

A complete trace of a native program is not really possible — the address space is opaque, the ISA is enormous, and the program can call into code you cannot see. WebAssembly removes each of those obstacles by construction:

- **Memory is one explicit, bounded region.** Not an address space with holes and mappings — a linear array with a known length that only grows through `memory.grow`. Every access is an index into it.
- **The instruction set is small and fully specified.** Core wasm is a few hundred opcodes with defined semantics, including the edge cases that are undefined behaviour in C.
- **Execution is deterministic.** Same module, same inputs, same components → byte-identical trace. A recorded failure is worth keeping because it reproduces.
- **There is no ambient authority.** A guest reaches the outside world only through declared imports, which is precisely the seam an instrumenting runtime wants to own.

The cost of this substrate is that the heap disappears. In wasm the heap is not a primitive — it is a region of linear memory managed by whatever allocator the guest brought with it. This is why conventional wasm tooling can report that memory grew but never who asked for it. Section 07 is about closing that.

Architecture: the substitutable machine

The central idea, borrowed one level down from deterministic simulation testing.

FoundationDB and TigerBeetle test distributed systems by making the environment substitutable: replace the network, the clock and the disk with deterministic models you control, and a failure that would take a fleet and a month of bad luck becomes reproducible on one thread in milliseconds. A system is only as testable as its environment is replaceable.

TraceWasm applies that idea one level down. The thing being substituted is not the network between machines — it is the machine itself. Every component a guest can observe is injected rather than fixed, and each is an ordinary Rust trait you can implement.

SUBSTITUTABLE · TRAIT Memory + MemoryView The guest's entire address space. Four required methods; every sized accessor defaults in terms of them.	SUBSTITUTABLE · TRAIT ImportRegistry The host boundary. Generated from an <code>impl</code> block by <code>#[imports]</code>, signature-checked against the module.	SUBSTITUTABLE · TYPE PARAM VirtualMachine <code>Stack</code> or <code>Register</code>, fixed at compile time. Selects the instruction set, the frame representation and the lowering pass.
SUBSTITUTABLE · CONFIG Config Resource ceilings — call-stack depth, memory caps — enforced by the driver rather than by the host OS.	FIXED Frontend Validation and section parsing via <code>wasmparser</code>, shared by both machines.	FIXED DWARF resolution Debug info is diagnostic-only and degrades gracefully; it never changes execution.

FIG. 1 The seams. The first four are supplied by the embedder; the last two are shared infrastructure. Everything downstream of a choice is generic over it — `Module<V>`, `Instance<M, I, V>`, `FuncBody<Instr>`.

The generic signature is carried all the way through rather than erased at the boundary. A compiled module is `Module<V: VirtualMachine>`; an instance is `Instance<M: Memory, I: ImportRegistry, V: VirtualMachine>`. Nothing is boxed and nothing is dynamically dispatched on the hot path, so substitutability costs zero at runtime — the monomorphised interpreter for a counting memory is a different compiled interpreter from the one for a flat buffer.

DESIGN DECISION

`VirtualMachine` is a **sealed** trait — an embedder cannot add a third machine. The seal buys a bijection between the machine type and its instruction type (`Instruction::Vm = Self`), which is what lets `execute` name `Module<Self::Vm>` without threading a fourth parameter through every signature. Memory and imports are deliberately *not* sealed; those are the seams meant for you.

Memory, in full

The flagship example of the generic design – and the reason the project exists.

A guest's only path to its own address space is `MemoryView`. There is no back door, no raw slice handed out, no fast path that bypasses the trait. That is what makes the trait's observations a *complete* account rather than a sample: if the program touched memory, it went through your code.

The interface is deliberately tiny. Four methods are required; every sized accessor — `read_u32`, `write_f64`, `fill`, and the rest — has a default implementation written in terms of those four, and can be overridden individually when a model needs to see width and alignment rather than bytes.

```
// Counts every guest read. Delegates everything else.
impl MemoryView for CountingMemory {
    fn read(&self, offset: usize, data: &mut [u8]) -> Result<(), MemoryError> {
        self.reads.set(self.reads.get() + 1);
        self.inner.read(offset, data)
    }
    fn write(&mut self, offset: usize, data: &[u8]) -> Result<(), MemoryError> { /* ... */ }
    fn copy_within(&mut self, dest: usize, src: usize, len: usize) -> Result<(), MemoryError> { /* ... */ }
}

fn size_in_bytes(&self) -> usize { self.inner.size_in_bytes() }
}

// Then instantiate against it. Nothing else in the program changes.
let mut instance = module.instantiate:::<CountingMemory, _>(NoImports, None)?;
```

FIG. 2 A memory model is an ordinary Rust type. The guest binary is untouched and unaware.

The split between `MemoryView` and `Memory` is a capability split, not an organisational one. Host functions receive only `MemoryView` — they can read and write but cannot resize, because the module's declared maximum and the instance's configured cap are policy that lives on the `Instance`. `Memory` adds `allocate_initial_memory` and `grow`, and only the runtime calls them.

WHAT THIS MAKES BUILDABLE

Cache-hierarchy memory

IN PROGRESS

Model L1/L2/L3 with a line size, associativity and replacement policy. Every guest read and write is already a tagged event arriving at your model, so miss rate, access pattern, locality and false sharing fall out directly — for hardware you do not own and have not built.

mmap-backed memory

IN PROGRESS

Back the region with a real mapping and use guard pages so a bounds check becomes a fault rather than a branch, or persist the region across runs to snapshot and resume guest state.

Shadow / taint memory

FALLS OUT OF THE SAME TRAIT

Keep a parallel byte-for-byte array of metadata — initialised-ness, provenance, taint labels — updated on every write and checked on every read. This is what MSan needs a compiler pass to achieve.

Adversarial memory

FALLS OUT OF THE SAME TRAIT

Make `grow` fail on the third call. Return `MemoryError` on a specific page. Deterministic fault injection at the exact granularity the guest can observe, with no fault-injection framework involved.

Two machines, one module

Tracing and speed pull in opposite directions, so the engine provides both and lets you choose per run.

The **stack machine** mirrors WebAssembly's own semantics one-to-one. Every operand push and pop is a real, observable event — which is exactly what you want when the question is *what did this program actually do*. Its `StackInstruction` enum has 192 variants, asserted at compile time to be at most 16 bytes.

The **register machine** lowers the same module ahead of execution, resolving operands to frame slots and eliminating the stack traffic the format implies but the program does not need.

`local.get; local.get; i32.add` becomes one instruction instead of three. Its `RegInstruction` enum has 184 variants in **8 bytes** — half the stack machine's footprint *despite* carrying explicit operands, because constants, locals and memory offsets are interned into the frame rather than stored inline.

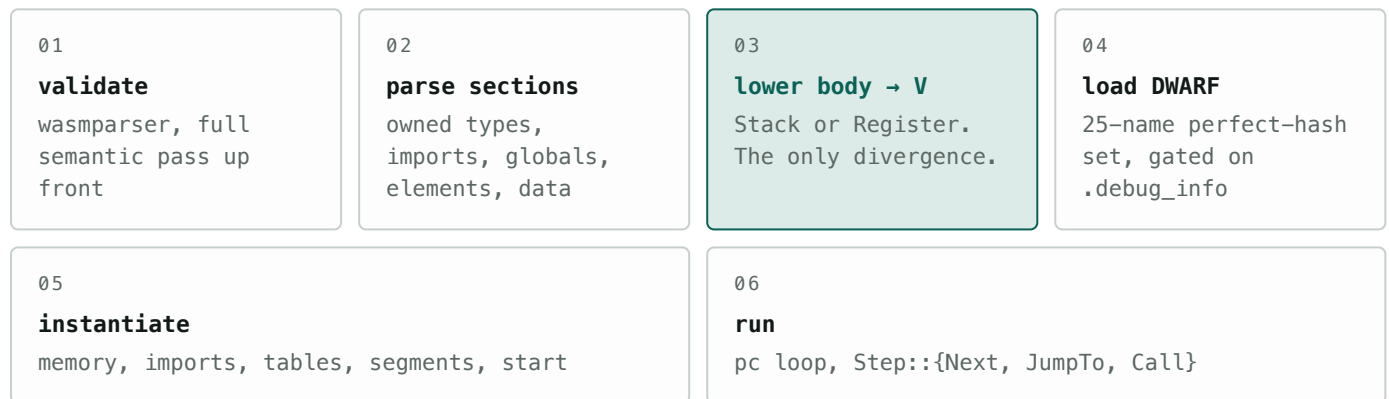


FIG. 3 The pipeline. Validation is a separate full pass, so the building pass never has to check anything — it only builds. The two machines share every stage but one.

Because validation happens up front and completely, the lowering pass is pure construction. And because `instruction_offsets` is kept parallel to `instructions` index-for-index, every lowered instruction can name the byte offset of the operator it came from — which is what makes source-level traps possible in section 08.

```
// The machine is a type parameter, fixed when the module is compiled.
let fast  = Module::<Register>::compile(&wasm)?;
let traced = Module::<Stack>::compile(&wasm)?;

// Same bytes, same exports, same results. Different machine.
let f = fast.get_typed_func::<(i32,), (i32,) >("double")?;
let mut inst = fast.instantiate::<LinearMemory, _>(NoImports, None)?;
assert_eq!(f.call((21,), &mut inst)?, (42,));
```

FIG. 4 Choosing a machine is a turbofish. Both compiled modules coexist; `TypedFunc` itself is machine-agnostic.

Register lowering: how the stack disappears

The compiler content. A single forward pass with deferred index assignment, plus three optimisations that fall out of the frame layout.

ONE FLAT INDEX SPACE, FOUR REGIONS

A register frame is four consecutive regions in one flat array. A `Slot` is a single untagged `u16` absolute frame index, so reading an operand is `registers[base + index]` — one add and one load. Only the region *sizes*, stored once in the frame layout, record which region a slot came from.

locals Params first, then declared locals. Addressable directly; never moved.	<code>base + 0</code>
consts The body's interned constant pool, materialised once at frame entry.	<code>base + locals</code>
spills Snapshots of locals whose value was borrowed before a write. Usually zero-sized.	<code>base + locals + consts</code>
operand registers Working slots. A callee is based here, at its caller's <code>caller_base</code> .	<code>base + locals + consts + spills</code>

FIG. 5 Constants and spills sit *below* the operand registers deliberately. A callee's frame begins at an operand-register index in the caller, so anything that must survive a call has to live below that point.

PROVISIONAL SLOTS AND A SINGLE BACKPATCH PASS

The region sizes are not known until the body ends — you cannot know how many constants a function interns until you have seen all of it. Rather than take a second pass over the instruction stream, lowering walks a *provisional* index space, emits every operand as the sentinel `Slot(u16::MAX)`, and records what each one actually is:

```
enum BackPatchableSlot {
    Register(u16),           // → shift up by consts + spills
    Spill(SpillIndex),       // → locals + consts + slot
    Const(InternedId<Const, u16>), // → locals + id
    Slot(Slot),              // already final – a local
}
```

FIG. 6 `BackpatchMap::apply` walks this once at end of body. One instruction may record several runs under one key – `br_if` records a condition and a move; `call_indirect` records a callee slot and its arguments – disambiguated by a source label and consumed positionally.

THREE OPTIMISATIONS THAT THE LAYOUT MAKES CHEAP

Lazy local forwarding

LOCAL ELISION

`local.get n` emits **nothing**. The operand records "read local `n`" and the consumer reads it in place, so `local.get; local.get; i32.add` costs zero copies.

A `local.set` or `local.tee` on a borrowed local first emits a `LocalSpill`, copying the old value somewhere the write cannot reach. One shared entry per borrowed value, ref-counted by live stack slots — so multiple simultaneous borrows and multiple live snapshots of the same local fall out of the representation. The design is modelled on Lua's upvalue closing.

Constant interning

NOT CONSTANT FOLDING

`i32.const` and friends emit nothing either. The value is interned into a per-body pool and pushed as a `Const` stack slot; the pool is materialised into the frame at entry, so a constant operand is just a frame index like any other.

Folding is deliberately *not* done here. `ref.is_null` on a known-null reference is decidable at lowering time and is still emitted — folding is left to whatever optimises the stream, so the lowering pass stays a single honest translation.

Operands inside the instruction

8-BYTE BUDGET

`InputRegisters<const I>` holds `I` inline slots; `OutputRegisters<const O>` holds a single start index, because destinations are always consecutive. That keeps the enum at 8 bytes.

Seven shapes do not fit — `if`, `br_if`, `br_table`, `call_indirect`, `select`, `memory.init` and `Move` — and carry a 4-byte arena id instead. A `memarg` offset is a full `u32`, so it is interned too; the ubiquitous offset `0` costs one pool entry for the whole body.

CONTROL FLOW

Block parameters and results must land in the same registers on every path, so one `Move` is emitted before each block exit. For `br_if` the move is a *field* of the instruction rather than a preceding one, so it executes only on the taken path.

Branch lowering never disturbs the simulated stack: truncation registers are found by peeking at depth rather than unwinding. That is what lets every arm of a `br_table` compute against the same stack state, and stops an outward branch from discarding the enclosing block's slots, borrows and spills. Dead code is filtered before the match — everything between an unconditional branch and the enclosing `else / end` is dropped, so blocks opened in unreachable code never reach the real control stack.

NEW FAILURE MODES, BY CONSTRUCTION

Packing operands into 16-bit indices introduces two errors the stack machine cannot produce: `RegisterFrameTooLarge` when a frame or region outgrows a `u16`, and `ToManyUniqueValues` when a body interns more constants or memory offsets than a 16-bit id can name. Both pools therefore use a fallible `try_intern` rather than the panicking one — the limits are real and are surfaced as compile errors, not as miscompilation.

Performance, and how it was actually measured

An optimisation campaign on the stack machine's hot path, with the negative results kept in.

A tracing interpreter that is too slow changes the behaviour it is trying to observe, so the hot path got a dedicated campaign. The figures below were produced by re-running the committed suite on an Apple M3 Max in release, not read out of a document.

WORKLOAD (STACK MACHINE)	PER CALL	NS / OP
arithmetic – i64 + f64 mix	1.20 ms	59.96
control flow – match + loops	1.63 ms	81.27
memory – load / store	1.23 ms	61.65
calls – one indirect per iter	839.90 µs	41.99
heap – alloc + collections	6.75 ms	–
trivial host call, n=1	122 ns	122.77
5-param host call	86 ns	86.34
compile + instantiate, 1.42 MB guest	396.14 µs	–

FIG. 7 Re-measured from the committed suite: `cargo test --release -p tracewasm-test --test metrics -- --nocapture --test-threads=1`. Guests are built for `wasm32-unknown-unknown -O` and link `std`, hence the ~1.4 MB module sizes.

WHAT ONE "OP" IS – AND IS NOT

An op is **one pass of a guest's `while i < n` body**, on the order of 10–30 wasm instructions — not one instruction. The suite says so in its own source and adds: *never read these figures as cycles-per-instruction*.

That caveat is load-bearing. Dividing by an assumed instruction count and an assumed clock yields a cycles-per-instruction number that can swing threefold depending on which end of the 10–30 range you pick, so no such figure appears in this document. Nothing in the engine counts *executed* instructions today — `instruction_count()` is a static count of instructions in a module. A per-instruction claim needs a retired-instruction counter first.

WHERE THE WINS CAME FROM

The single largest lever was inlining the per-instruction `execute` into the dispatch loop: **–40% to –44%** across the suite, larger than every other change combined. It is not free — the driver frame grew from 416 B to 5,088 B, cutting reachable guest depth from roughly 6,400 frames to 1,650, which is why `Config::max_call_stack_depth` exists and defaults to 2,000.

HISTORICAL – NOT RE-MEASURABLE

Across the whole campaign the suite moved roughly $2.5\times$: arithmetic $142.89 \rightarrow 58.39$ ns/op, memory $177.33 \rightarrow 61.13$, calls $116.57 \rightarrow 44.51$, control flow $138.55 \rightarrow 83.39$, heap $16.01 \rightarrow 6.33$ ms. The "after" column reproduces today within noise, which is what gives the deltas their weight — but the "before" column is a historical record from earlier commits and cannot be re-measured without reverting them.

The second lever was *where function boundaries sit relative to the register allocator*. A boundary is a barrier the allocator cannot see across, so the rule runs in both directions: put boundaries **between** hot and cold code, and **never inside** a hot loop.

`Stack::push` had `Vec::push` inlined into it, which dragged `RawVec::grow_one` along. Instrumented over one 20,000-iteration workload, the growth path ran **9 times out of 580,021 pushes** — 0.0016%. But because the grown value was live across that call, it needed a callee-saved register, and the save had to precede the branch. The prologue supporting a path taken nine times ran 580,021 times.

MOVING THE GROWTH PATH BEHIND <code>#[inline(never)]</code>	BEFORE	AFTER
Common-path instructions	18	9
Register save/restore accesses	6	0
Callee-saved registers before the branch	3	0
arithmetic / memory / frames	146 / 176 / 116 ns	-5.4% / -9.9% / -7.2%

FIG. 8 Interleaved A-B-A-B, not AAA-BBB — the first attempt reported a 14% win that was partly thermal drift. Every sample of the fast variant beat every sample of the slow variant in every round.

FOOTPRINT

The stack machine's `Instruction` went from 40 bytes to 16, a $2.5\times$ reduction enforced by `const _: () = assert!(size_of::<Instruction>() <= 16)`. Across the six guest modules that is 85,908 instructions: **1,342 KB instead of 3,355 KB**. Notably, this produced *no measurable throughput change* — it is a footprint win, and it is reported as one.

THE NEGATIVE RESULTS ARE THE CREDIBILITY

Three separate `#[inline]` experiments produced no measurable change. Hoisting loop invariants produced no measurable gain. A `&mut pc` and boxed-error refactor gave zero throughput improvement. Outlining the heaviest cold arm — the `call_indirect` mismatch path, which expands `core::fmt` twice — recovered **32 bytes of frame, 0.6%**, because two identical-looking frame costs have different causes: callee-saved registers held across a call are removable by outlining, spill slots spread over ~ 200 match arms are not.

Three tempting explanations were refuted outright. It was not the branch: pre-filling the backing store to make it perfectly predictable moved throughput not at all, and made per-call latency **363× worse**. It was not code size: `execute` was byte-identical before and after. And `#[cold]` was consistently *worse*, by 3–5%.

REGISTER VERSUS STACK

The two machines are compared by a dedicated harness that runs them on identical modules with **interleaved, order-flipped rounds** — never all of one then all of the other, because an earlier measurement in this repo reported a 14% win that turned out to be partly thermal drift. Each row is the median of seven rounds, and every row reports how many of those seven the winner actually took.

WORKLOAD	STACK	REGISTER	SPEEDUP	ROUNDS WON
arithmetic – i64 + f64 mix	64.30 ns	39.21 ns	1.64×	7 / 7
memory – load / store	61.10 ns	41.08 ns	1.49×	7 / 7
control flow – match + loops	86.39 ns	70.38 ns	1.23×	7 / 7
heap – alloc + collections	6.34 ms	5.26 ms	1.20×	7 / 7
calls – one indirect per iter	44.33 ns	40.12 ns	1.10×	7 / 7
trivial host call	125.41 ns	106.18 ns	1.18×	7 / 7
5-param host call	87.93 ns	75.28 ns	1.17×	7 / 7

FIG. 9 `cargo test --release -p tracewasm-test --test machine_comparison`. The spread is the point: the register machine's advantage is removing operand traffic, which is most of what arithmetic does and least of what a call does — so 1.64× on arithmetic and 1.10× on calls is the expected *shape*, not an inconsistency. Before any timing, the harness asserts both machines return bit-identical results; a speed comparison between machines that disagree would be meaningless.

The footprint difference is larger than the speed difference, and unlike timing it is exact and host-independent — so the harness asserts on it rather than merely reporting it.

COMPILED FOOTPRINT	STACK	REGISTER	RATIO
instruction size	16 B	8 B	2.00×
instruction stream, six guests	1,342 KB	377 KB	3.55×

FIG. 10 The stream shrinks further than the instruction does, because register lowering also emits *fewer* instructions: `local.get` and `i32.const` become operands rather than instructions, so they disappear from the stream entirely.

WHAT IS DELIBERATELY NOT ASSERTED

No threshold says the register machine must be faster. It is, on every row, in every round — but encoding that as a bound would make the suite fail on a loaded CI box, which trains everyone to ignore it. The assertions are the ones that hold regardless of host speed: the two machines agree on every result, no row measured zero time, and the footprint ordering holds exactly.

The host boundary, and seeing the heap

Where the guest meets the outside world is a seam you own — including the ability to make it fail.

A procedural macro turns an ordinary `impl` block into an `ImportRegistry`, checking each method's signature against what the guest actually declares. Ask for a `MemoryView` parameter and the guest's memory is handed to you. Return a `Result` and a failure becomes a trap in the calling frame rather than a panic.

```
struct Host { calls: u32 }

#[imports]
impl Host {
    #[module("env")]
    fn log(&mut self, value: i32) -> (i32,) {
        self.calls += 1;
        (value,)
    }
}
```

FIG. 11 Signature mismatches against the guest's declared import are a compile-time or instantiation-time error, not a runtime surprise.

WHY THIS CLOSES THE HEAP PROBLEM

In WebAssembly the heap is not a primitive — it is a region of linear memory managed by the allocator the guest brought with it. A tracer watching linear memory sees bytes changing and can report that memory grew; it cannot report who asked, how much, with what alignment, or when it was released.

The in-progress answer is to make the allocator an *import*. Link the guest against an allocator whose `alloc`, `realloc` and `dealloc` cross back into the host as ordinary imported calls, and every allocation event becomes first-class — with its size, its alignment, its lifetime, and, via the DWARF machinery in the next section, the exact source line responsible.

Fragmentation, leaks, short-lived churn and allocation hot spots stop being statistics and become specific lines of code.

Traps that point at your source

DWARF resolution, including the inlined frames optimisation folded away.

When a wasm module traps, what you are normally handed is an offset into a binary — true, and nearly useless. Programs compiled from Rust carry DWARF describing the source they came from, and WebAssembly encodes DWARF code addresses as byte offsets into the module binary, which is exactly what the interpreter already records per instruction.

So the reconstruction is direct. A trap captures a `TraceRecord` per active frame;

`to_source_trace()` builds one `addr2line` context for the whole trace — indexing is proportional to DWARF size, so doing it per frame would re-parse everything — and resolves each frame's offset into file, line and demangled function name, **expanding inlined frames** that optimisation had folded away.

- Debug sections are recognised by a 25-name perfect-hash set matching `gimli`'s own section names, and routed out of the generic custom-section bucket.
- Loading is gated on `.debug_info` actually being present, because the loader returns `Ok` with an empty slice for absent sections — without the gate every module would report debug info it did not have.
- A malformed debug section is swallowed rather than failing the compile. Debug info is diagnostic-only; it degrades traces, it never changes execution.
- Every wasm frame yields exactly one source record even with no DWARF coverage, so the source trace never silently drops a frame relative to the wasm trace.

Measured end to end: a Rust `panic!` inside a guest surfaces as an `unreachable` trap carrying a full **10-frame** call chain, resolvable to source. A failure deep inside a dependency reads the way it would have if the program had never been compiled to WebAssembly at all.

The AOT tier: an LLVM IR builder with no LLVM in it

In progress. What exists today is the IR-construction half of a backend — and it is deliberately stricter than LLVM.

The ahead-of-time tier is being built on a from-scratch LLVM IR layer: 15,464 lines of Rust, 199 tests, and **zero LLVM dependencies**. No `inkwell`, no `llvm-sys`, no `build.rs`, no `llvm-config`. It constructs IR in memory and renders textual `.ll`; it neither parses IR nor links against `libLLVM`.

That choice buys a clean build with no native toolchain in the dependency graph, and it makes the IR a plain owned data structure that can be inspected, walked and tested like any other Rust value. What it costs is that turning `.ll` into machine code is an external step — that step does not exist yet.

THE INTERESTING PART

The builders are **deliberately stricter than `llvm-as`**. An out-of-range constant GEP array index and a `load` whose stated type disagrees with the inferred pointee are both legal LLVM, and both are rejected here. Types are interned, so structural equality is an integer comparison and a type cannot even print itself without its pool. Terminators take the cursor *by value*, so appending after a terminator does not compile — there is a `compile_fail` doctest pinning that.

BUILT TODAY

- **19 instruction kinds, 21 builder methods**, each validated and each with an emitter arm — phi, branches, ret, alloca, load, store, GEP, call, icmp/fcmp, integer and float binops, fneg, cast, switch, select, unreachable.
- **Full predicate coverage**: 10 integer conditions, 16 float conditions, 13 integer binops, 5 float binops, 12 cast opcodes. Explicit optional alignment on load, store and alloca.
- **Type system**: opaque pointers (LLVM-15 style), arrays, packed and unpacked structs, function types, the full integer and float set — all interned.
- **Operand typing** distinguishes *asserted* from *inferred*. Inference walks a pointer back to its defining instruction to reconstruct a pointee; assertion is a check, never a coercion, so a mismatch is an error rather than a silent bitcast.
- **Module level**: target triple and data layout, 11 linkage kinds, 3 visibilities, global variables with and without initialisers, a tpestate-tagged global id distinguishing defined functions, declared functions and globals.
- **A visitor layer** (`CfgVisitor`) with one hook per instruction kind. The text emitter is simply one implementation of it; a verifier or an optimiser would be another.

NOT BUILT YET — STATED PLAINLY

- **The wasm → IR lowering**. The seam exists as an `Emitter` trait, and its sole implementation in the core crate is a `todo!()`. This is the main body of remaining work.
- **Any path from `.ll` to machine code**. No object emission, no JIT, no `llc` invocation.

- **Metadata and debug info.** No `!dbg`, no named metadata — which matters, because the tracing story depends on DWARF surviving the AOT tier.
- **Attributes and instruction flags** (`nsw`, `nuw`, `exact`, `fast-math`), atomics, vectors, aggregates by value, `invoke/landingpad`, intrinsics.
- **Automated `llvm-as` verification.** Emitter tests are golden strings that were checked against `llvm-as` by hand at authoring time. Nothing in CI runs it. Two `todo!()` panics remain on the indirect-call path.

Calling this an AOT backend today overstates it. It is the IR-construction half of one, built first and built carefully, because a lowering pass is far easier to write against a layer that rejects your mistakes than against one that prints them.

Correctness: rustc is the oracle

Differential testing against natively compiled Rust, so there are no hand-written expected values to be wrong.

The testing strategy was deliberately re-scoped. The WebAssembly spec testsuite is 257 `.wast` files and roughly 52,600 `assert_return` directives — but only about 53 of those files are pure-MVP, and a large fraction of the invalid-module assertions test the *parser*, not the engine. Meanwhile an audit of what rustc actually emits found the post-MVP surface to be tiny and already fully implemented: sign extension, saturating truncation, `memory.copy`, `memory.fill`, and untyped `select`. Multi-value is enabled in the toolchain but zero functions in any tested binary return more than one value.

So the primary tier is differential testing against the native build. Each guest source file is compiled *twice* — once to `wasm32-unknown-unknown` by the build script, and once natively via path-included modules — and the two results are compared. rustc's own backend is the oracle, which means no magic constants and no test that can be wrong in the same direction as the implementation.

GUEST MODULE	LINES	EXERCISES
arithmetic	417	div/rem edges, shifts & rotates, bit counting, float min/max, rounding, NaN specials, saturating conversions, reinterpret
control_flow	398	nested loops, dense and sparse <code>br_table</code> , direct and mutual recursion, iterator chains, <code>?</code> , labelled blocks
heap	396	Vec growth, sort/dedup, strings, HashMap, BTreeMap, VecDeque, BinaryHeap, smart pointers, drop order, churn
memory	364	narrow loads/stores, endianness, unaligned access, overlapping copy, fill, growth, OOB and div-zero traps
frames	355	many locals, 5-param calls, call chains, indirect recursion, closure nesting, dyn dispatch, monomorphisation
exotic	607	enum payloads, Option/Result chains, slice patterns, UTF-8, casts, <code>#[repr]</code> layouts, traits, custom Ord

FIG. 12 Six guest modules, 152 exported functions, run under *both* machines and against native Rust. Plus 12 checked-in `.wasm` fixtures driving 62 numeric edge-case tests. Two documented exclusions: transcendentals (host libm and compiled-in libm differ in the last bit) and hash iteration order.

A realistic program using `Vec`, `sort_unstable`, `dedup`, `HashMap`, `BTreeMap`, `format!`, `&dyn Debug`, floating-point math and iterator chains returns the same answer under both machines and natively, under both `-O` and `-C debuginfo=2`.

A BUG ONLY THIS APPROACH FINDS

Deep guest recursion aborted the process at 2,000–4,000 wasm frames — `fatal runtime error: stack overflow`, a `SIGABRT` rather than a catchable error, because one native frame backs each wasm frame. Fixed with an explicit configurable depth ceiling. The same investigation produced a methodology warning worth repeating: a depth test appeared to survive two million frames because LLVM had turned the accumulator recursion into a loop. Tree and indirect recursion do not get flattened; accumulator recursion does.

Status, without rounding up

Unsupported features are rejected at compile time with a named error, never mis-executed.

WORKING	IN PROGRESS	REJECTED AT COMPILE TIME
Stack VM – 192 variants, ≤ 16 B	Wasm $\rightarrow$ LLVM IR lowering (<code>todo!()</code>)	SIMD / <code>v128</code>
Register VM – 184 variants, ≤ 8 B	Machine-code emission from <code>.ll</code>	Threads & atomics
Generic <code>Memory</code> / <code>MemoryView</code>	Cache-hierarchy memory model	The component model
Generic <code>ImportRegistry</code> + <code>#[imports]</code>	mmap-backed memory	64-bit memories and tables
Module compile & instantiate	Imported allocator for heap tracing	More than one memory per module
Sign extension, saturating truncation	Trace recording & query surface	Table instructions (<code>table.*</code>, <code>elem.drop</code>)
Bulk memory: copy / fill / init, <code>data.drop</code>	Subscribable execution events	Exception handling, tail calls
DWARF source traces, inlined frames	Async VM, yielding at imported calls	GC types (refused in the type section)
Differential suite vs. native Rust	Declarative benchmark harness	Non-funcref tables; non-func/global imports
Configurable depth & resource ceilings		
LLVM IR builder + textual emitter		

FIG. 13 The right-hand column is a design position, not a gap list. A module using any of it fails with `TraceWasmError::Unsupported` naming the operator – the engine declines rather than guessing, because a tracing tool that silently mis-executes is worse than one that refuses.

The scope discipline comes from the target: TraceWasm is built to run wasm compiled from real source programs, with Rust as the first-class case. The surface rustc actually emits is small, and the engine covers it completely. Nothing about the design is Rust-specific, though — any language that targets WebAssembly and carries standard debug information works unchanged, because the interpreter has no notion of the source language beyond the DWARF it is handed.

Where it goes

Ordered by what unblocks the most downstream work.

NEXT

Declarative benchmark harness

Benchmark any region of guest code by annotation, rather than only at whole-export granularity as the current comparison harness does.

NEXT

Event subscription

`instruction_start`, `instruction_complete`, `frame_pushed`, `frame_popped`. Turns the interpreter from something you substitute components into, to something you subscribe to.

NEAR

Cache-hierarchy memory

The flagship demonstration of the generic memory design: real miss rates and locality analysis for a machine described rather than owned.

NEAR

Imported allocator

Drop-in `alloc` / `realloc` / `dealloc` via binary transformation, making every heap event observable and attributable to a source line.

ONGOING

Async VM and executor

Yield at imported calls so host functions can be genuinely asynchronous, without one native frame pinned per guest frame.

LARGER

Wasm → LLVM lowering, then codegen

The AOT tier proper: implement the `Emitter` seam against the existing IR layer, add debug metadata so traces survive the tier, then an external path to machine code.

The through-line is that none of these are separate products. An event stream, a cache model, an allocator and an AOT tier are all the same architectural bet — that if the machine is a parameter, the interesting tools are things you plug in rather than things that have to be built into the engine.